scipy sparse matrix division

Asked 5 years, 8 months ago Modified 1 year, 5 months ago Viewed 10k times

4

I have been trying to divide a python scipy sparse matrix by a vector sum of its rows. Here is my code

```
sparse_mat = bsr_matrix((l_data, (l_row, l_col)), dtype=float)
sparse_mat = sparse_mat / (sparse_mat.sum(axis = 1)[:,None])
```

However, it throws an error no matter how I try it

```
sparse_mat = sparse_mat / (sparse_mat.sum(axis = 1)[:,None])
File "/usr/lib/python2.7/dist-packages/scipy/sparse/base.py", line 381, in __div__
return self.__truediv__(other)
File "/usr/lib/python2.7/dist-packages/scipy/sparse/compressed.py", line 427, in
__truediv__
raise NotImplementedError
NotImplementedError
```

Anyone with an idea of where I am going wrong?

python numpy scipy sparse-matrix Edit tags

Share Edit Follow Close Flag

asked Feb 14, 2017 at 11:42

uchman21

668 2 6 22

The division calls the true_division, which is an element-wise division. This seems not to be implemented for more than one value. So, most probably, the result of `(sparse_mat.sum(axis = 1)[:,None]` is not a single number. – Dschoni Feb 14, 2017 at 11:56

@Dschoni Yes, the result is a vector and my aim is to divide each element in each row of the sparse matrix with the sum of the row elements. So if M=[[2,4],[1,2]], I want to get Ans=[[2/6, 4/6],[1/3, 2/3]]. – uchman21 Feb 14, 2017 at 12:11

Have you tried `sparse_mat = sparse_mat*(1 / (sparse_mat.sum(axis = 1)[:,None]))` . It seems the division of sparse matrices is the problem. You may also have to convert the divisor to a dense array `sparse_mat = sparse_mat*(1 / (sparse_mat.sum(axis = 1).toarray()[:,None]))` – Daniel F Feb 14, 2017 at 12:12

@uchman21 Please provide a small self contained example. This problem might be related to the data you put into the matrix. (Or it might be that your scipy is too old - sparse matrix division as I tried it works on Python 3 and scipy 0.18.) – MB-F Feb 14, 2017 at 12:20

I am using python 2.7.13 with scipy 0.18. The matrix is just a simple sparse matrix of 232 x 232 – uchman21 Feb 14, 2017 at 16:11

3 Answers

Sorted by:
Reset to default

Date modified (newest first)

2

Per this message, to keep the matrix sparse, you access the *data* values and use the (nonzero) indices:

```
sums = np.asarray(A.sum(axis=1)).squeeze()  # this is dense
A.data /= sums[A.nonzero()[0]]
```

If dividing by the nonzero row mean instead of the sum, one can

```
nnz = A.getnnz(axis=1)  # this is also dense
means = sums / nnz
A.data /= means[A.nonzero()[0]]
```

Share Edit Follow Flag

edited May 25, 2021 at 15:25

answered May 24, 2021 at 23:57

user394430
2,665 2 26 27

3

Something funny is going on. I have no problem performing the element division. I wonder if it's a Py2 issue. I'm using Py3.

```
In [1022]: A=sparse.bsr_matrix([[2,4],[1,2]])
In [1023]: A
Out[1023]:
<2x2 sparse matrix of type '<class 'numpy.int32'>'
    with 4 stored elements (blocksize = 2x2) in Block Sparse Row format>
In [1024]: A.A
Out[1024]:
array([[2, 4],
       [1, 2]], dtype=int32)
In [1025]: A.sum(axis=1)
Out[1025]:
matrix([[6],
        [3]], dtype=int32)
In [1026]: A/A.sum(axis=1)
Out[1026]:
matrix([[ 0.33333333,  0.66666667],
        [ 0.33333333,  0.66666667]])
```

or to try the other example:

```
In [1027]: b=sparse.bsr_matrix([[0, 9, 0, 0, 1, 0],
      ...:                      [2, 0, 5, 0, 0, 9],
      ...:                      [0, 2, 0, 0, 0, 0],
      ...:                      [2, 0, 0, 0, 0, 0],
      ...:                      [0, 9, 5, 3, 0, 7],
      ...:                      [1, 0, 0, 8, 9, 0]])
In [1028]: b
Out[1028]:
<6x6 sparse matrix of type '<class 'numpy.int32'>'
    with 14 stored elements (blocksize = 1x1) in Block Sparse Row format>
```

```
In [1029]: b.sum(axis=1)
Out[1029]:
matrix([[10],
        [16],
        [ 2],
        [ 2],
        [24],
        [18]], dtype=int32)
In [1030]: b/b.sum(axis=1)
Out[1030]:
matrix([[ 0.        ,  0.9       ,  0.        ,  0.        ,  0.1       , 0.        ],
        [ 0.125     ,  0.        ,  0.3125    ,  0.        ,  0.        , 0.5625    ],
 ....
        [ 0.05555556,  0.        ,  0.        ,  0.44444444,  0.5       ,     0.
]])
```

The result of this sparse/dense is also dense, where as the `c*b` (`c` is the sparse diagonal) is sparse.

```
In [1039]: c*b
Out[1039]:
<6x6 sparse matrix of type '<class 'numpy.float64'>'
    with 14 stored elements in Compressed Sparse Row format>
```

The sparse sum is a dense matrix. It is 2d, so there's no need to expand it dimensions. In fact if I try that I get an error:

```
In [1031]: A/(A.sum(axis=1)[:,None])
....
ValueError: shape too large to be a matrix.
```

Share Edit Follow Flag

edited Feb 14, 2017 at 17:43

answered Feb 14, 2017 at 17:31

hpaulj
209k 13 213 320

It seems to depend on the scipy version. Using an outdated version, this actually worked as expected for me where the division of two sparse vectors returned a sparse vector. After all, if the dividend has an empty cell, result should be 0 in this cell anyway. For a newer version of scipy, the same line returns a dense numpy matrix... – Radio Controlled Mar 20, 2019 at 9:24

9

You can circumvent the problem by creating a sparse diagonal matrix from the reciprocals of your row sums and then multiplying it with your matrix. In the product the diagonal matrix goes left and your matrix goes right.

Example:

```
>>> a
array([[0, 9, 0, 0, 1, 0],
       [2, 0, 5, 0, 0, 9],
       [0, 2, 0, 0, 0, 0],
       [2, 0, 0, 0, 0, 0],
       [0, 9, 5, 3, 0, 7],
       [1, 0, 0, 8, 9, 0]])
```

```
>>> b = sparse.bsr_matrix(a)
>>>
>>> c = sparse.diags(1/b.sum(axis=1).A.ravel())
>>> # on older scipy versions the offsets parameter (default 0)
... # is a required argument, thus
... # c = sparse.diags(1/b.sum(axis=1).A.ravel(), 0)
...
>>> a/a.sum(axis=1, keepdims=True)
array([[ 0.        ,  0.9       ,  0.        ,  0.        ,  0.1       ,  0.        ],
       [ 0.125     ,  0.        ,  0.3125    ,  0.        ,  0.        ,  0.5625    ],
       [ 0.        ,  1.        ,  0.        ,  0.        ,  0.        ,  0.        ],
       [ 1.        ,  0.        ,  0.        ,  0.        ,  0.        ,  0.        ],
       [ 0.        ,  0.375     ,  0.20833333,  0.125     ,  0.        ,  0.29166667],
       [ 0.05555556,  0.        ,  0.        ,  0.44444444,  0.5       ,  0.        ]])
>>> (c @ b).todense() # on Python < 3.5 replace c @ b with c.dot(b)
matrix([[ 0.        ,  0.9       ,  0.        ,  0.        ,  0.1       ,  0.        ],
        [ 0.125     ,  0.        ,  0.3125    ,  0.        ,  0.        ,  0.5625    ],
        [ 0.        ,  1.        ,  0.        ,  0.        ,  0.        ,  0.        ],
        [ 1.        ,  0.        ,  0.        ,  0.        ,  0.        ,  0.        ],
        [ 0.        ,  0.375     ,  0.20833333,  0.125     ,  0.        ,  0.29166667],
        [ 0.05555556,  0.        ,  0.        ,  0.44444444,  0.5       ,  0.
]])
```

Share Edit Follow Flag

edited Feb 14, 2017 at 13:31

answered Feb 14, 2017 at 12:34

Paul Panzer
50.7k 2 46 94

I tried this solution but it give out error elem_sum = csc_matrix((1/sparse_mat.sum(axis = -1).A.ravel(), numpy.arange(sparse_mat.shape[0]), numpy.arange(sparse_mat.shape[0]+1))) File "/usr/lib/python2.7/dist-packages/scipy/sparse/compressed.py", line 548, in sum return spmatrix.sum(self,axis) File "/usr/lib/python2.7/dist-packages/scipy/sparse/base.py", line 629, in sum raise ValueError("axis out of bounds") ValueError: axis out of bounds – uchman21 Feb 14, 2017 at 13:08

@uchman21 Strange, try `axis = 1` then, that seemed to work in your code. – Paul Panzer Feb 14, 2017 at 13:14

yes that work. However, for me I needed to make it something like c = sparse.diags(1/b.sum(axis=1).A.ravel(),0) to specify the main diagonal for it to finally work. Please add that to you answer. – uchman21 Feb 14, 2017 at 13:21

2 `b.sum(axis=1).A1` should work. The `sum` produces a np.matrix ,which has a A1`property. stackoverflow.com/a/20765358/901925 – hpaulj Feb 14, 2017 at 15:05